



TUTO6: DEVELOPING HIERARCHICAL VISUALIZATIONS FROM AGGLOMERATIVE CLUSTERING OF TEXT

NICOLAS MÉDOC LUXEMBOURG INST. OF SCIENCE AND TECHNOLOGY

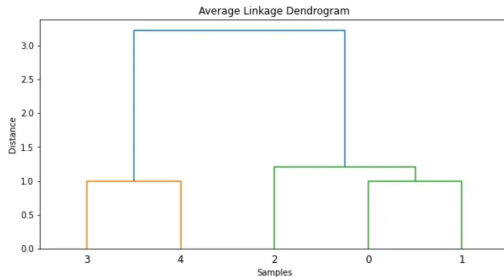


Outline

1. Agglomerative clustering from text data
2. Building tree layout with D3JS
3. Building a dendrogram
4. Building an icicleplot
5. Building a treemap
6. Multi-resolution clustering by cutting the tree



Agglomerative clustering



Clustering using Average Linkage Method

Grouping clusters based on a linkage function:

- Single: nearest neighbor
- complete: longest distance between any two points in the two clusters
- average: average distance between all pairs of points in the two clusters
- ward: minimizing the increase in variance



Getting Python Flask Server from git

<https://github.com/nicolasmedoc/TD5-text-Python>

git checkout agglomerative

In Server.py:

```
...
import clustering
import dendrogram
...
cluster_tree = clustering.agglomerative(None,x_lsa,"ward","euclidean",distance_threshold=
cluster_tree_dict = dendrogram.get_tree_dict(cluster_tree, true_k, dataset.target)
...
@app.route("/getClusterTree")
def get_cluster_tree():
    return {"cluster_tree": cluster_tree_dict, "categories": dataset.target.tolist(), "di
```

Test: <http://localhost:5000/getClusterTree>



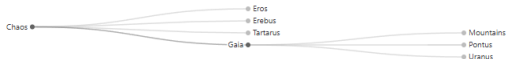
D3Js: hierarchical layouts

<https://d3js.org/d3-hierarchy>

- D3JS provides predefined functions for hierarchical layouts. They compute the position/size of nodes and links.
- available layouts:



D3Js: hierarchical layouts



d3.tree: node/link position (x,y)



d3.cluster: node/link position (x,y)



d3.treemap: rectangles (x0,y0,x1,y1)



d3.partition: rectangles (x0,y0,x1,y1)



D3Js: hierarchical layouts

Need a tree structure (e.g., `data.cluster_tree`):

```
{
  node_id:0,
  count_by_true_categ:[215,150,312,198], // number of leaves in each category
  children:[ // it's intermediate node in the tree
    {
      node_id: 1,
      count_by_true_categ: [0,0,1,0],
      true_category: 2, // the category of the document
      children: [] // it's a leaf (document)
    },
    ...
  ]
}
```



D3Js: hierarchical layouts

<https://d3js.org/d3-hierarchy>

- `d3.hierarchy`: create a root node to prepare the data for the layout computation
- It returns root and all descendants with the following attributes:
 - `node.data`: the node object as passed to `hierarchy`
 - `node.depth`: zero for the root, increasing by one for each descendant generation
 - `node.height`: the greatest distance from any descendant leaf, or zero for leaves
 - `node.parent`: the parent node, or null for the root node
 - `node.children`: an array of child nodes, if any, or undefined for leaves
 - `node.value` - the optional summed value of the node and its descendants (after calling `sum()` or `count()`)



D3Js: building a dendrogram

<https://github.com/nicolasmedoc/TD5-text-React>
git checkout tree

components/hierarchy-d3.js: in computeLayout(), create d3.hierarchy nodes, and compute a dendrogram layout with d3.cluster()

```
const hierarchyLayout = d3.hierarchy(visData.cluster_tree);  
if(this.layout==="dendrogram"){  
  // creation of the layout function  
  const cluster = d3.cluster().size([this.width,this.height]);  
  // call cluster function with hierarchy in parameter  
  // to update nodes in hierarchyLayout object  
  cluster(hierarchyLayout); // compute node.x and node.y  
}
```



D3Js: building a dendrogram

components/hierarchy-d3.js: in nodeLinkBasedLayout(), create a circle for each node

```
const descendants = hierarchy.descendants();
this.hierSvg.selectAll(".node")
  .data(descendants,(d)=>d.data.node_id)
  .enter().append("g")
  .attr("class", function(d) { return "node"; })
  .attr("transform", function(d) { return "translate(" + d.x + "," + d.y + ")"; })
  .append("circle")
  .attr("r", 1)
  ;
```



D3Js: building a dendrogram

Exercise 1: components/hierarchy-d3.js: in nodeLinkBasedLayout(), create a line for each link from each non-root node to its parent (line attributes: x1,y1,x2,y2)

```
const descendants = hierarchy.descendants();
this.hierSvg.selectAll(".node")
  .data(descendants,(d)=>d.data.node_id)
  .enter().append("g")
  .attr("class", function(d) { return "node"; })
  .attr("transform", function(d) { return "translate(" + d.x + "," + d.y + ")"; })
  .append("circle")
  .attr("r", 1)
  ;
```



D3Js: building a dendrogram

The order of nodes is not optimal. To control it, d3.hierarchy provides a sort method. For dendrograms, a common approach is to sort by node heights (nb of levels from the node to the deeper leaf).

In computeLayout:

```
if(this.layout==="dendrogram"){  
  hierarchyLayout.sort((a, b) => b.height - a.height ||  
                           d3.ascending(a.node_id, b.node_id)  
                           );  
  ...  
}
```



D3Js: building an icicleplot

components/hierarchy-d3.js: in computeLayout(), call d3.partition(). This methods rely on node.value to compute the size of each node rectangles. One approach is to count the number of leaves in the subtree of each node:

```
}else if (this.layout==="icicle"){  
  hierarchyLayout.count();  
  hierarchyLayout.sort((a, b) => b.height - a.height || b.value - a.value);  
  const partition = d3.partition().size([this.width,this.height]);  
  partition(hierarchyLayout) // compute rectangles  
  // in each node: (x0,y0) top left of the rectangle,  
  //                (x1,y1) bottom right  
}...
```



D3Js: building an icicleplot

Exercise 2: components/hierarchy-d3.js: in rectangleBaseLayout() create a rectangle for each node.

Change the layout in components/hierarchy-container.js:

```
// config.layout = "dendrogram";  
config.layout = "icicle";  
// config.layout = "treemap";
```



D3Js: building a treemap

Exercise 3: components/hierarchy-d3.js: in computeLayout() create a d3.treemap layout

Change the layout in components/hierarchy-container.js:

```
// config.layout = "dendrogram";  
// config.layout = "icicle";  
config.layout = "treemap";
```



Multi-resolution clustering

How to cut the tree to search clusters at different levels of granularity?



Multi-resolution clustering

How to cut the tree to search clusters at different levels of granularity?

- by choosing a level: each subtree being a cluster
- by searching node with a certain distance between sub-trees



Multi-resolution clustering

The Python method `getClusterTree()` returns an additional attribute "distance" which contains for each node, the largest distance between the leaves of the subtree.

The function `clusterByCut()` in `HierarchySlice` exploits this information to search the first node with a distance $<$ threshold, and mark it as a cluster node (in attribute `cluster_dist_cut`), and propagates it to the subtree.

- **Exercise 4:** In `applyColor()` function, color the nodes (and the links by using `cluster_dist_cut`), the list of clusters are stored in `visData.clusters` (to parameter the `colorScale`)

to compare the two color encoding comment/uncomment in `HierarchyContainer.js`:

```
config.colorEncoding="distancecut"  
// config.colorEncoding="majoritycateg"
```



Multi-resolution clustering

You can change the threshold in `HierarchySlice` `getClusterTree()` by varying `cuttingRate` between `]0,1]`.

- **Exercise 5:** add a reducer action to update this rate and update the `cluster_dist_cut` based on a distance threshold passed in `action.payload`
- **Exercise 6:** in `ConfigBar` add two combo lists to control the layout and the `colorEncoding`. In case of `colorEncoding==="clustercut"` show a slider to control the distance threshold.